

Semantic Knowledge Representation and Reasoning: A First-Order Logic Approach for Explainable AI

Submitted by:

Hemang Joshi

Eashaan Thakur

Submitted to:

Dr. Rajakrishnan P Rajkumar

Project Codebase:

github.com/karma-skz/CL2-NLxFOL-parser-and-reasoning-module (refactor branch)

Department of Computational Linguistics
International Institute of Information Technology, Hyderabad

December 5, 2025

Acknowledgement

We would like to express our heartfelt gratitude to **Dr. Rajakrishnan P Rajkumar** for his continuous guidance and support throughout the development of this project. His expertise in logic, computational linguistics, and AI explainability deeply shaped our direction and encouraged us to pursue rigorous design choices.

We also extend our sincere thanks to our TA **Ekansh Goyal** for their timely assistance, constructive feedback, and constant support during the course of this work.

Abstract

Modern AI systems, particularly large neural models, often behave as black boxes, making it difficult to understand or audit their decision-making processes. To address this, we present a complete pipeline for translating natural language (NL) sentences into First-Order Logic (FOL), validating symbolic expressions through multiple parsing backends, and performing symbolic reasoning using both Datalog-style inference and formal proof systems. The system integrates a rule-driven NL $\rightarrow$ FOL translator, a multi-validator FOL parsing layer, a persistent knowledge base with entailment tracing, and two formal reasoning engines: a Natural Deduction prover and a classical resolution prover.

Beyond implementation, this report emphasizes a metrics-oriented evaluation: (i) syntactic validity of generated FOL across validators, (ii) translation accuracy on NL $\leftrightarrow$ FOL datasets, and (iii) entailment correctness in the knowledge base and proof engines. We analyze strengths (deterministic, interpretable translation and tracing) and limitations (coverage gaps, strictness mismatch across validators, and domain sensitivity), providing a clear path for improvement.

Contents

1	Introduction	6
2	Background and Related Work	8
3	Research Questions and Proposed Solution	10
4	Motivation	12
5	System Design and Architecture	13
5.1	Design Principles	13
5.2	Architectural Overview	13
5.3	Translation Pipeline Architecture	14
5.4	Validator Architecture	14
5.5	Knowledge Base Architecture	14
5.6	Resolution Prover Architecture	15
5.7	Natural Deduction Architecture	15
5.8	Benchmarking Architecture	15
5.9	System Data Flow	15
5.10	Strengths and Limitations	16
6	Implementation Details and Component Breakdown	17
6.1	Command-Line Interface (CLI)	17
6.2	Translation Subsystem	17
6.3	NL Methods and External Parsers	18
6.4	Resolution Engine	19
6.5	Natural Deduction Proof System	19
6.6	Knowledge Base and Traceable Inference	20
6.7	Benchmarking and Evaluation	20
6.8	Testing and Documentation	20
6.9	Interfaces and Data Contracts	21
6.10	Typical Invocation Patterns	21
6.11	Resolution Logic and Formal Justification	22

7	Evaluation Methods	25
7.1	FOLIO Dataset	25
7.2	Benchmark Systems	26
7.2.1	Internal Translator Benchmark	26
7.2.2	Gemini LLM Benchmark	26
7.3	Comparison and Reporting Tools	27
7.4	Metrics	27
7.5	Gold-Standard Test Suite	27
7.5.1	Structure and Purpose	27
7.5.2	Execution and Artifacts	28
7.6	Benchmark Artifacts	28
7.7	Run Examples	28
8	Results	30
8.1	Translation Coverage	30
8.2	Translation Quality Metrics	30
8.3	Validator Agreement	31
8.4	Gold-Standard Reasoning Tests	32
8.4.1	Pass/Fail Outcomes	32
8.4.2	Error Breakdown	32
8.4.3	Query Accuracy	32
8.4.4	Timing Analysis	33
8.5	Run Progress and Stability	33
8.6	Summary of Findings	34
8.7	Qualitative Results: Sample Inputs, Outputs, and Traces	34
8.7.1	Sample Inputs	35
8.7.2	Sample Outputs	35
8.7.3	Derivation Traces in the Knowledge Base	36
8.7.4	Resolution and Proof Traces	36
8.7.5	Log Files and Diagnostics	37
8.7.6	Comparison Reports	38
8.7.7	Reproducibility Commands	38
9	Analysis and Discussion	39
9.1	Translation Coverage and Quality	39
9.2	Validator Behaviour and Divergence	39
9.2.1	FoMaster Validator	40
9.2.2	Lark Validator	40
9.2.3	NLTK Validator	40
9.2.4	Which Validator Was Best?	40
9.3	Reasoning Performance and Error Sources	40
9.4	Performance of the Resolution Engine	41
9.4.1	Why Resolution Worked Reliably	41

9.4.2	Fit with Dataset and System Design	41
9.4.3	Comparison with KB Reasoning	42
9.5	What Went Right	42
9.6	What Went Wrong	42
9.7	Overall Interpretation	43
9.8	Summary	43
10	Conclusion and Future Work	44
10.1	Conclusion	44
10.2	Future Work	45
10.2.1	Expanding Rule and Template Coverage	45
10.2.2	LLM-Guided Translation with Stronger Syntax Control	46
10.2.3	A Hybrid Translator: Rule-Based First, LLM as Fallback	46
10.2.4	Normalization Pipeline for Validator Harmonization	46
10.2.5	Enhanced Reasoning Beyond Definite Clauses	47
10.2.6	Dataset-Augmented Rule Learning	47
10.3	Closing Remarks	47

Chapter 1

Introduction

Explainable AI (XAI) has become increasingly important as AI systems are now used in sensitive areas such as healthcare, finance, public policy, and law. In these settings, it is not enough for an AI system to produce a correct answer. It must also be able to show how it arrived at that answer. However, the most common machine learning models today, especially large neural networks, do not provide clear reasoning steps. Their internal structure is complex, and it is difficult to understand how their predictions are generated. This lack of transparency creates problems related to trust, accountability, and the ability to audit decisions.

A practical way to improve explainability is to use symbolic reasoning. Symbolic representations make the reasoning steps explicit and easy to inspect. First-Order Logic (FOL) is well suited for this purpose because it represents knowledge in a structured form and uses clear rules for inference. When information is encoded in FOL, each conclusion can be traced back to the rules or facts used to derive it. This makes it possible for users to understand and verify the reasoning process.

The aim of this project is to build a complete system that brings these advantages of symbolic logic into the domain of natural language reasoning. The system takes English sentences as input, translates them into FOL, validates the logical structure, stores the resulting formulas in a knowledge base, and performs reasoning tasks such as answering queries or generating proof steps. This pipeline provides a clear and interpretable alternative to black-box AI systems.

To achieve this, the project includes several components that work together. A rule-based translation module identifies linguistic patterns and maps them to logical templates. Multiple validators check whether the generated FOL expression is syntactically correct. A knowledge base stores facts and rules and is able to trace how conclusions are reached. A Natural Deduction proof engine and a separate resolution prover allow users to verify results through two different formal reasoning methods. By combining all these tools, the system can support explainable reasoning from natural language input.

Although symbolic systems offer strong benefits for transparency, they face several challenges. Rule-based translation can fail when sentences do not match the expected patterns. Natural language is highly variable, and even simple phrases can have multiple interpretations. As a result, the system may produce incomplete or incorrect logical forms if the rule set does not cover a specific construction. Another challenge is that symbolic reasoning requires well-formed input. If a sentence is translated incorrectly, the entire reasoning process may break down.

There are several possible ways to improve these issues. The rule base can be expanded and refined by examining common failure cases. Error messages from the validators can be made more informative so that users can quickly identify and fix problems. Hybrid approaches are also promising. For example, lightweight NLP tools can help identify sentence structure or choose more accurate rules during translation. Another option is automatic rule suggestion based on common sentence patterns in datasets.

This project focuses on building a modular and transparent reasoning pipeline that can be extended over time. By converting natural language into logic and providing formal tools for inference, the system demonstrates a practical way to make reasoning processes more explainable. It also highlights how symbolic methods can complement modern AI approaches and address some of the limitations of current black-box models.

Chapter 2

Background and Related Work

Symbolic AI has a long history of using logic-based methods to represent and reason about knowledge. Before the rise of large neural models, many natural language understanding systems relied entirely on structured rules, formal grammars, and logic-driven inference. These approaches focused on making the reasoning process clear and reproducible, which remains a central goal of Explainable AI today. The ideas developed in earlier symbolic systems continue to influence modern research, especially in domains where interpretability and formal guarantees are important.

A key area of related work is semantic parsing. Traditional semantic parsers used hand-crafted grammars, lexicons, and templates to convert natural language sentences into formal representations such as First-Order Logic. These systems were relatively easy to inspect because each rule directly reflected how a specific linguistic structure should be interpreted. More recently, neural semantic parsers have been introduced. They can generalize to a wider range of inputs, but their internal decision-making is difficult to understand. This reduces their usefulness in applications where a clear mapping from language to logic is required. Our project builds upon ideas from early rule-based parsers, using pattern matching and templates to create logical forms in a controlled and interpretable way.

Another relevant area of research is Explainable AI. There has been growing interest in building AI systems that can justify their outputs rather than simply producing them. Many studies emphasize the need for transparent chains of reasoning so that users can verify how a conclusion was reached. Logic-based formalisms such as FOL naturally support this requirement because they express knowledge through explicit premises and rules. When an inference is made, the system can point to the exact statements that contributed to the result. This ability is particularly valuable in settings such as legal reasoning, auditing, and education. The system developed in our project follows this principle by storing logic statements in a knowledge base and producing detailed traces of how each query is evaluated.

Several established reasoning frameworks also contribute to the foundation of this work. Classical resolution is widely used in automated theorem proving and provides a systematic procedure for determining whether a formula is satisfiable. Natural Deduction, on the other hand, models reasoning in a way that is closer to human logical argumentation. It offers introduction and elimination rules for logical connectives, making proofs easier to understand. Datalog and logic programming provide a declarative approach to knowledge representation,

especially for rule-based systems. These frameworks demonstrate different styles of reasoning, and our project incorporates both Natural Deduction and resolution in order to compare how the same logical information can be processed in different ways.

Datasets for logic-based natural language tasks have gained attention in recent years. FO-LIO provides pairs of English sentences and their corresponding FOL translations, along with tasks involving consistency and entailment. LogicNLI focuses on testing reasoning abilities by providing natural language inputs that must be transformed into logical relations such as contradiction or entailment. These datasets highlight the challenges of translating between natural language and formal representations. They also provide structured benchmarks for evaluating how well a system handles translation and inference. The design of our translation and validation components is informed by the difficulties observed in such datasets, especially the importance of consistent syntax and reliable rule selection.

Overall, the work in semantic parsing, XAI, symbolic inference frameworks, and logic-based datasets forms the foundation of our project. We bring together these ideas into a single modular workflow that can translate natural language into logic, validate the results, store and process them through a knowledge base, and reason about them using formal proof systems. By combining these research directions, the project aims to create a system that is both technically robust and easy to understand, making it suitable for educational use as well as practical experimentation in explainable reasoning.

Chapter 3

Research Questions and Proposed Solution

This project is guided by several research questions that focus on improving the transparency, auditability, and reliability of AI systems by grounding their reasoning in formal logic. Modern AI systems often produce outputs without showing the reasoning behind them, which makes it difficult to verify or challenge their conclusions. Our work explores how symbolic methods can address these issues and how a complete pipeline can be built to support explainable reasoning from natural language input.

Research Questions

Q1. How can semantic parsing improve explainability and auditability of AI systems by translating natural language into verifiable First-Order Logic (FOL) representations?

Semantic parsing allows natural language statements to be converted into structured logical forms. When these forms are expressed in FOL, each part of the meaning becomes explicit and can be checked for correctness. This project investigates how a rule-based translation approach, supported by well-defined templates and syntactic validation, can produce logical formulas that are both understandable and verifiable. By turning ordinary language into logic, the system makes the reasoning process transparent and easy to audit.

Q2. How can rule-based parsing ensure accurate and interpretable FOL translations?

Rule-based parsing provides a controlled and deterministic mapping from linguistic patterns to logical structures. Unlike statistical models, rule-based systems do not rely on hidden representations. Each translation step is tied to a specific rule, which directly supports interpretability. This project examines how a rule library, combined with pattern matching and templating, can reliably handle common sentence types while providing clear explanations for how each logical form was produced. Multiple validators further ensure that the output is syntactically sound before it is used in reasoning.

Q3. How does symbolic reasoning enhance traceability compared to black-box

models?

Symbolic reasoning frameworks, such as Datalog, Natural Deduction, and resolution, allow every inference step to be traced back to exact facts and rules. This stands in contrast to neural models, where the reasoning process is encoded in complex internal states that cannot be inspected. Our project explores how a symbolic knowledge base and formal proof systems can make reasoning fully traceable. When a query is answered, the system records which rules were used, how intermediate conclusions were derived, and why the final answer holds. This traceability forms the foundation of explainable logic-based AI.

Solution Provided

The system developed in this project offers a unified solution to these research questions by building a complete pipeline for explainable reasoning:

- A rule-based natural language to FOL translator that produces structured logical formulas.
- Multiple validation methods that check syntax and ensure that each FOL expression is correct and well-formed.
- A knowledge base that stores facts and rules and supports transparent, traceable reasoning through a pyDatalog environment.
- A Natural Deduction proof engine that shows inference steps in a human-readable way.
- A resolution prover that applies a classical logical method to verify entailment.

By integrating these components, the system demonstrates how natural language can be converted into formal logic, how these logical forms can be checked and stored, and how reasoning can be performed in a way that is easy to understand, audit, and verify. This provides a practical and transparent alternative to black-box AI systems, addressing the core research questions through a modular and interpretable symbolic framework.

Chapter 4

Motivation

Neural models, despite their capabilities, remain opaque. Domains such as legal reasoning, medical diagnosis, and regulatory auditing require explicit explanations, not just predictions. Symbolic logic offers:

- **Transparency:** Reasoning steps are explicit and human-readable.
- **Traceability:** Every inference can be linked to a concrete rule or fact.
- **Verifiability:** Logical derivations can be independently checked.

By building a complete symbolic pipeline, this project aims to create a foundation for explainable AI systems with rigorous formal properties.

Chapter 5

System Design and Architecture

The system is designed as a modular and extensible pipeline that translates natural language into First-Order Logic, validates the resulting formulas, and performs symbolic reasoning through multiple inference engines. The overall architecture emphasizes four design principles: modularity, determinism, explainability, and extensibility. Each subsystem operates independently, exposes a clear interface, and contributes to a fully traceable reasoning workflow.

5.1 Design Principles

Modularity: The system is divided into separate layers for translation, validation, knowledge management, reasoning, and benchmarking. Each layer is responsible for a specific part of the workflow, allowing components to evolve independently without affecting the rest of the system.

Determinism: The NL→FOL translator uses rule-driven pattern matching and template-based generation. This ensures predictable and repeatable outputs for the same input sentence and prevents hidden or probabilistic transformations.

Explainability: Explainability is built into every stage. Validators report explicit syntax errors, the knowledge base produces derivation traces, the Natural Deduction engine outputs step-by-step proofs, and the resolution prover provides classical refutation paths. Benchmark outputs further document translation behavior across datasets.

Extensibility: The architecture supports pluggable translation rules, validator backends, and inference engines. New NL→FOL methods, proof systems, or datasets can be added with minimal integration work due to the clean subsystem boundaries.

5.2 Architectural Overview

The system is composed of six major subsystems:

1. NL→FOL Translation Pipeline
2. Multi-Backend FOL Validators
3. Knowledge Base with Logic Tracing

4. Resolution Prover
5. Natural Deduction Proof System
6. Benchmarking and Evaluation Framework

These subsystems interact in a linear yet flexible pipeline where translation and validation always precede reasoning, and benchmark tools may attach at any stage to measure system performance.

5.3 Translation Pipeline Architecture

The translation pipeline begins with text normalization and clause segmentation, followed by NLP preprocessing to extract structural information. Rule matching identifies a suitable logical template, which is then instantiated into an FOL fragment. Validity checks are applied both at the fragment level and at the final combined expression. If translation succeeds, the formula is passed to reasoning modules or stored in the knowledge base.

This design ensures that translation errors are caught early, enabling clear feedback and reliable downstream reasoning.

5.4 Validator Architecture

Validators operate as independent backends, each supporting a different interpretation of FOL syntax. The system integrates:

- a strict grammar-driven Lark validator,
- an NLTK-based logic parser with normalized notation,
- a lightweight FO-Master validator for resilience against minor irregularities.

Validator independence allows cross-checking of outputs to detect ambiguous or unstable translations. The architecture supports adding new validators by simply registering a new parsing method.

5.5 Knowledge Base Architecture

The knowledge base is a persistent, rule-driven environment that supports adding facts, defining rules, and evaluating logical queries. It uses a pyDatalog-backed model where all facts and rules are maintained in a shared logic environment. Queries produce both boolean outcomes and detailed derivation traces, enabling users to inspect how a conclusion was reached.

The KB architecture separates storage, inference, and tracing, making it easy to integrate with external tools, reset state, or swap in alternative reasoning engines.

5.6 Resolution Prover Architecture

The resolution subsystem follows the classical refutation-based reasoning model. It is structured around a pipeline that converts FOL input into CNF, standardizes variables, and iteratively resolves clauses to detect contradictions. The prover operates both as a standalone reasoning engine and as a baseline for comparing results from the KB and Natural Deduction systems.

Its file-driven interface is intentionally decoupled from the rest of the logic environment, allowing clean batch processing of queries.

5.7 Natural Deduction Architecture

The Natural Deduction subsystem parses logical statements into an abstract syntax tree and applies inference rules step by step. This design mirrors textbook manual proofs and supports fine-grained inspection of each inference. The architecture separates parsing, rule application, and proof state management, making the ND system transparent and educationally useful.

5.8 Benchmarking Architecture

The benchmarking framework is built around dataset loaders, translation runners, and metric aggregators. It supports both internal translation and external model baselines, such as LLM-generated FOL. Benchmark outputs are stored in structured JSON or JSONL formats, enabling comparison across runs and detection of regressions. The framework is extensible to new datasets, metrics, and evaluation modes.

5.9 System Data Flow

A unified data flow connects all system components:

1. Natural language input is normalized and parsed.
2. A translation rule is selected and instantiated into FOL.
3. Validators confirm syntactic correctness.
4. Valid formulas are:
 - stored in the knowledge base for traceable reasoning, or
 - passed to the resolution or ND prover for analysis.
5. Benchmarking tools capture translation success and reasoning correctness.

This structured flow ensures consistency across subsystems and makes it easy to trace how an input sentence progresses through the architecture.

5.10 Strengths and Limitations

Strengths:

- Clear separation of concerns across modular subsystems.
- Deterministic translation and controlled template-based logic generation.
- Strong explainability through validators, traces, and proof outputs.
- Availability of multiple reasoning engines for cross-validation.

Limitations:

- Finite rule sets limit translation coverage on unfamiliar sentence patterns.
- Validator strictness may differ, creating inconsistencies in borderline cases.
- Resolution may scale poorly on large clause sets due to CNF expansion.
- Templates optimized for benchmark datasets may miss free-form natural language variations.

Overall, the architecture provides a robust and extensible foundation for explainable reasoning while remaining easy to analyze, test, and expand.

Chapter 6

Implementation Details and Component Breakdown

This chapter expands on the internal components of the system and describes how each subsystem is implemented. Unlike the previous chapter, which focused on high-level design principles and architectural organization, this section explains the concrete modules, data structures, interfaces, and execution flow that make up the full $NL \rightarrow FOL \rightarrow Reasoning$ pipeline.

6.1 Command-Line Interface (CLI)

The primary entry point for user interaction is `cli/main.py`. The CLI reads natural language input (from a file or standard input), invokes the translator, optionally performs reasoning, and writes output to `cli/output.txt`. The control flow follows a simple orchestration pattern:

1. Parse CLI arguments.
2. Execute translation pipeline.
3. Run optional reasoning modules (KB, Resolution, ND).
4. Handle and format errors.

The IO format is intentionally lightweight and text-based to simplify debugging and testing.

6.2 Translation Subsystem

The $NL \rightarrow FOL$ translation module is the core of the system. It is implemented using several coordinated files:

- `translator/translator.py`: Main controller for translation.
- `translator/rules.yaml`: Declarative mapping of NL patterns to FOL templates.
- `translator/matcher.py`: Rule selection based on structural pattern matching.
- `translator/templates.py`: Template expansion into FOL fragments.

- `translator/nlp.py`: Tokenization, POS tagging, normalization, and clause segmentation.
- `translator/data_structures.py`: Defines internal types for tokens, clauses, and FOL fragments.

Translation Flow

The translation pipeline proceeds through the following stages:

1. **Normalization and segmentation**: Handles conditionals (“if ... then ...”) and conjunctions (“and”).
2. **NLP preprocessing**: Tokenize and assign POS tags while preserving proper noun casing.
3. **Rule matching**: Identify the most appropriate rule pattern from `rules.yaml`.
4. **Template filling**: Construct FOL fragments by substituting extracted arguments into templates.
5. **Validation**: Use `translator/fol_validator.py` for both per-fragment and final FOL checks.

Output Contract

The translator returns either:

- A valid FOL string, or
- A structured error object describing unmatched rules or invalid expressions.

This contract simplifies integration with validators, resolution, and benchmarking tools.

6.3 NL Methods and External Parsers

The system includes a dedicated interface for running NL→FOL conversions through alternate methods:

- `nl_methods/nl_to_fol.py`: Executes a single-pass translation on batches of facts and implications.

This is primarily used for preparing input to the resolution engine.

FO-Master Method

The FO-Master subsystem includes lightweight tools for FOL normalization:

- `adapter.py`: Normalizes operators (e.g., `,`, `=>`, `&`, `|`).
- `fol_atomic.py`: Models predicates, terms, and simple FOL shapes.

- `fol_syntax_semantics.py`: Checks arity, scoping, and connective correctness.
- `validator.py`: Performs regex-based tokenization and structural checks.

Lark and NLTK Validators

- **Lark Validator**: Provides strict EBNF-style grammar enforcement.
- **NLTK Validator**: Normalizes notation and uses `nltk.sem.logic` for semantic parsing.

Each validator serves as an independent check to ensure final FOL expressions are well-formed.

6.4 Resolution Engine

The resolution subsystem is implemented through the following components:

- `engine.py`: Main resolution pipeline.
- `statement.py`: Clause representation, unification rules, tautology detection.
- `predicate.py`: Functor names, terms, and hashing semantics.

Resolution Workflow

1. Parse FOL formulas.
2. Eliminate implications ($\Rightarrow$).
3. Apply negation rules (De Morgan).
4. Standardize variables and perform Skolemization if needed.
5. Convert to CNF using distributive rules.
6. Iteratively resolve clause pairs using set-of-sets management.
7. Detect empty clause to signal contradiction (entailment).

The engine reads from `nl_methods/input.txt` and writes to `nl_methods/output.txt`.

6.5 Natural Deduction Proof System

The ND subsystem supports human-readable proofs and includes:

- `proof/parser.py`: Converts formulas into an AST.
- `proof/ast_nodes.py`: Node types for formulas and derivation steps.
- `proof/engine.py`: Applies ND rules with variable freshness and scoping checks.

- `proof/nd_prover.py`: Coordinates proof construction and outputs readable traces.

The ND system can operate on FOL expressions produced by the translator or provided manually.

6.6 Knowledge Base and Traceable Inference

The KB engine is implemented in `knowledge_base/engine.py` and supports:

- Loading and saving facts and rules via `kb_store.json`.
- Direct fact assertions and rule applications.
- Scoped queries across stored predicates.
- Derivation logging for explainable reasoning.

The persistence layer uses JSON to store predicates, entities, and rule structures.

6.7 Benchmarking and Evaluation

The benchmarking subsystem evaluates translation quality and system behavior across datasets:

- `benchmarks/run_nlmmodel.py`: Runs the internal translator on the FOLIO dataset.
- `benchmarks/run_gemini.py`: Runs translations using the Gemini LLM for comparison.
- `benchmarks/metrics.py`: Aggregates translation success rates and summary statistics.
- `benchmarks/report_translation_comparison.py`: Compares gold-standard FOL with generated FOL.

Outputs appear in `outputs/folio` as JSONL rows and summary files.

6.8 Testing and Documentation

The project includes a full testing and documentation suite:

- `tests/run_test_suite.py`: Executes system-level tests defined in `tests/test.json`.
- `docs/benchmark_commands.md`: Lists runnable commands and usage patterns.
- `.vscode/settings.json`: Provides an optimized development configuration.

6.9 Interfaces and Data Contracts

- **FOL Format:** Standard predicate syntax $P(x,y)$, connectives $\neg$, $\&$, $\vee$, $\Rightarrow$, and quantifiers where supported.
- **Validator Contract:** Given a formula, return pass/fail with normalized output or diagnostics.
- **Resolution Contract:** Given clauses and query, return entailment result and intermediate clause sets.
- **Benchmark Row Schema:** Includes sentence ID, NL sentence, gold FOL, model metadata, produced FOL, and error fields.

6.10 Typical Invocation Patterns

- **Resolution input:** `nl_methods/input.txt` containing facts, rules, and queries.
- **Gemini runner:** Automatic retry with exponential backoff and strict JSON parsing.
- **Translator matching:** Rule selection through priority scoring in `matcher.py` with template parameterization.

6.11 Resolution Logic and Formal Justification

The resolution prover included in this system implements a classical method for first-order logical inference. Resolution is a rule of refutation-based reasoning that determines whether a conclusion follows from a set of premises. The method operates on formulas converted into Conjunctive Normal Form (CNF) and repeatedly applies the resolution rule to derive new clauses. If an empty clause is eventually produced, the system has derived a contradiction, which proves that the original conclusion is entailed by the premises.

Resolution Workflow

The resolution engine follows these steps:

1. **Convert all input formulas into CNF.** This involves removing implications, pushing negations inward using De Morgan's laws, standardizing variables, and distributing disjunction over conjunction.
2. **Negate the query.** To check whether a query Q follows from a knowledge base KB , resolution proves that $KB \cup \{\neg Q\}$ is inconsistent.
3. **Initialize a clause set with all CNF clauses.** Each clause is a disjunction of literals.
4. **Iteratively apply the resolution rule.** For any two clauses C_1 and C_2 , if one contains a literal L and the other contains its negation $\neg L$, they may be resolved to produce a new clause.
5. **Check for the empty clause.** If the procedure derives the empty clause $\square$, a contradiction has been found.
6. **Determine satisfiability.** Producing the empty clause means the negated query contradicts the premises, so the original query is entailed.

This workflow allows the resolution prover to serve as a classical and fully formal reasoning method that complements the knowledge base's Datalog-style inference.

The Resolution Rule

Formally, the resolution rule states:

$$\frac{(A \vee L), (\neg L \vee B)}{A \vee B}$$

Here, L is any literal, and $A \vee B$ is called the *resolvent*. The rule removes a complementary pair L and $\neg L$ and merges the remaining literals.

When applied systematically across all clauses, this rule explores the logical consequences of the formula set.

Why Resolution Works

Resolution is widely used because it satisfies two essential logical properties for first-order inference: **soundness** and **refutational completeness**. These properties ensure that the algorithm is both reliable and powerful enough to decide entailment.

Soundness

A rule is sound if every conclusion derived from it is logically valid. Resolution is sound because:

- The resolvent of two clauses is true in every model where both original clauses are true.
- Removing complementary literals does not introduce new information, it only eliminates contradictions.
- The resolvent represents the weakest clause that must hold whenever the original pair holds.

Formally, if both premises are satisfied in a model, the resolvent must also be satisfied in that model. This ensures that resolution never derives a false conclusion from true premises.

Refutational Completeness

Resolution is *refutationally complete*. This means that if a set of clauses is logically inconsistent, the resolution procedure is guaranteed to derive the empty clause after a finite number of steps.

Applied to entailment:

$$KB \models Q \quad \text{iff} \quad KB \cup \{\neg Q\} \models \perp$$

Resolution proves entailment by showing that adding the negation of the query leads to a contradiction.

Completeness relies on these facts:

- Every first-order formula can be converted into an equisatisfiable CNF form.
- For every contradiction in CNF, there exists a finite resolution derivation reaching the empty clause.
- The resolution closure of the clause set eventually contains all logical consequences relevant to the contradiction.

Thus, if the query truly follows from the knowledge base, resolution will eventually detect the inconsistency caused by its negation.

Why the Model in This System Uses CNF

Resolution operates on pairs of clauses, and clauses are defined only in CNF. Converting all statements into CNF ensures:

- A uniform structure for literals and clauses.
- A small, constant set of inference rules (only the resolution rule is needed).
- A predictable and systematic search space for contradictions.

Although CNF conversion may increase the number of clauses, it preserves satisfiability and keeps the reasoning process consistent across all inputs.

Role of Resolution in the Full System

The resolution prover complements the rule-based KB in several ways:

- It provides a classical baseline for logical entailment.
- It allows checking the correctness of KB-derived conclusions.
- It highlights inconsistencies in rules, translations, or stored formulas.
- It supports tasks where full first-order resolution is more expressive than Datalog inference.

Together with the Natural Deduction engine, the resolution module forms a complete symbolic reasoning suite that supports explainability, verification, and cross-validation of results produced by other components.

Chapter 7

Evaluation Methods

Evaluation of the NL→FOL translation system and the symbolic reasoning components relies on a combination of structured datasets, automated benchmarks, and a curated gold-standard test suite. This chapter describes the resources used for evaluation, the benchmark runners that process them, and the artifacts generated during experimentation.

7.1 FOLIO Dataset

The primary corpus used for assessing translation performance is **FOLIO**, located at `benchmarks/datasets/FOLIO`. FOLIO provides natural language premises and conclusions with optional gold-standard First-Order Logic representations. The dataset is stored in versioned JSONL files under:

- `benchmarks/datasets/FOLIO/data/v0.0`

Each split (such as `train` or `validation`) is represented as `folio-{split}.jsonl`. Each example includes fields that support grouping, tracking, and translation evaluation:

- **story_id** and **example_id**: Identifiers for grouping related sentences.
- **label**: Entailment type, typically entailment or contradiction.
- **premises**: Natural language sentences serving as context.
- **premises-FOL**: Optional list of gold FOL translations aligned with the premises.
- **conclusion**: The hypothesis to be evaluated.
- **conclusion-FOL**: Optional gold FOL translation for the conclusion.

Both the internal translator and the LLM-based runner iterate through each premise and conclusion, attach gold FOL when available, and record translation success or errors. FOLIO provides broad coverage of logical constructs, making it suitable for both translation accuracy and validator robustness studies.

7.2 Benchmark Systems

Benchmarking is performed using dedicated runners that execute translation tasks, validate outputs, and write structured evaluation artifacts.

7.2.1 Internal Translator Benchmark

The script `benchmarks/run_nlmodel.py` evaluates the deterministic rule-based translator on FOLIO.

Function:

- Runs NL→FOL translation using the system’s rule/template pipeline.
- Validates outputs using the available validators.
- Records per-sentence outcomes and an aggregate summary.

Outputs:

- JSONL rows (e.g., `outputs/folio/nlmodel_100rows.jsonl`)
- Summary JSON (e.g., `outputs/folio/nlmodel_100summary.json`)

These results allow inspection of coverage, error types, and syntactic validity across a large dataset.

7.2.2 Gemini LLM Benchmark

The LLM benchmark, implemented in `benchmarks/run_gemini.py`, evaluates model-produced FOL using Google’s `gemini-2.0-flash` (or a user-selected variant).

Method:

- Issues strict JSON prompts of the form `{“fol” : “...”}`.
- Removes Markdown fences and ensures the presence of a valid `fol` field.
- Uses exponential backoff and retry logging to handle rate limits and API instability.
- Captures raw model responses and cleaned outputs for diagnostics.

Outputs:

- `outputs/folio/gemini_rows.jsonl`
- `outputs/folio/gemini_summary.json`

This benchmark provides a comparison between deterministic translation and LLM-generated FOL, highlighting areas where rule-based translation is more or less reliable.

7.3 Comparison and Reporting Tools

The script `benchmarks/report_translation_comparison.py` provides qualitative and quantitative reports comparing model-produced FOL with gold-standard references.

Outputs:

- `outputs/folio/comparison.txt`: Full comparison of gold vs. predicted FOL.
- `outputs/folio/comparison_folio100.txt`: Sample-focused comparison file.

These reports support error analysis, predicate mismatch detection, and rule refinement.

7.4 Metrics

Metrics are computed using `benchmarks/metrics.py`, which aggregates:

- Total number of sentences processed.
- Number of successful, validated translations.
- Overall translation rate across the dataset.

These metrics provide simple yet effective indicators of rule coverage and system stability.

7.5 Gold-Standard Test Suite

In addition to FOLIO, the system is evaluated using a curated set of gold-standard tests located at `tests/test.json`. These tests provide deterministic, interpretable checks for both translation and reasoning.

7.5.1 Structure and Purpose

Each test case contains:

- Natural language or FOL inputs.
- A defined task (e.g., entailment check).
- An `expected` array specifying the correct outputs (e.g., `[true, false]`).

This suite complements dataset-based benchmarks by:

- Verifying inference correctness under controlled conditions.
- Detecting regressions in translation, validation, and reasoning.
- Providing hand-annotated, high-precision examples independent of dataset variability.

7.5.2 Execution and Artifacts

The test harness `tests/run_test_suite.py` loads the gold tests and evaluates system outputs. Results are written to `tests/test_results.json` for regression tracking.

7.6 Benchmark Artifacts

All benchmark runs produce a common set of artifacts:

Rows JSONL

Each line represents one sentence-level evaluation and includes:

- Identifiers (`story_id`, `example_id`, sentence type and index).
- The natural language sentence.
- Gold FOL (if provided).
- Translation or error message.
- Model metadata and optional debugging fields.

Summary JSON

Run-level aggregates that simplify comparison between multiple experiments.

Text Reports

Readable side-by-side comparisons of predicted and gold FOL for manual inspection.

7.7 Run Examples

Environment Setup

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Internal Translator

```
python benchmarks/run_nlmmodel.py --split validation --limit-examples 100
```

Gemini Benchmark

```
export GOOGLE_API_KEY=your_key
python benchmarks/run_gemini.py --split validation --limit-examples 100 \
--model gemini-2.0-flash
```

Comparison Report

```
python benchmarks/report_translation_comparison.py \  
  --rows outputs/folio/gemini_rows.jsonl \  
  --summary outputs/folio/gemini_summary.json
```

Gold-Standard Tests

```
python tests/run_test_suite.py
```

These commands represent typical workflows for evaluating the system under different configurations.

Chapter 8

Results

This chapter presents the empirical evaluation of the NL→FOL translation pipeline, the validator suite, and the symbolic reasoning engines. Results come from two main sources:

- Benchmarking on the FOLIO dataset using both the internal rule-based translator and the Gemini 2.0 Flash model.
- A curated gold-standard test suite evaluating reasoning correctness, inference traceability, and runtime performance.

We report translation coverage, structural quality metrics, validator agreement, and reasoning accuracy, accompanied by visual summaries and excerpts from the generated output files.

8.1 Translation Coverage

Figure 8.1 compares the number of sentences successfully translated by the rule-based parser and the Gemini 2.0 Flash model. According to the benchmark summary, the rule-based system translated 127 out of 338 sentences, yielding a coverage of **37.57%**. In contrast, Gemini produced translations for all 338 sentences, corresponding to **100%** coverage.

This gap reflects the limited rule coverage of the deterministic parser and highlights the LLM’s ability to generalize beyond the template set.

8.2 Translation Quality Metrics

Beyond coverage, we evaluate structural similarity between predicted and gold FOL using predicate-level F1, BLEU, and tree similarity. From the comparison outputs:

- **Rule-based predicate F1:** 0.2782
- **Gemini predicate F1:** 0.6360

with a corresponding improvement of **+0.358** according to the comparison report.

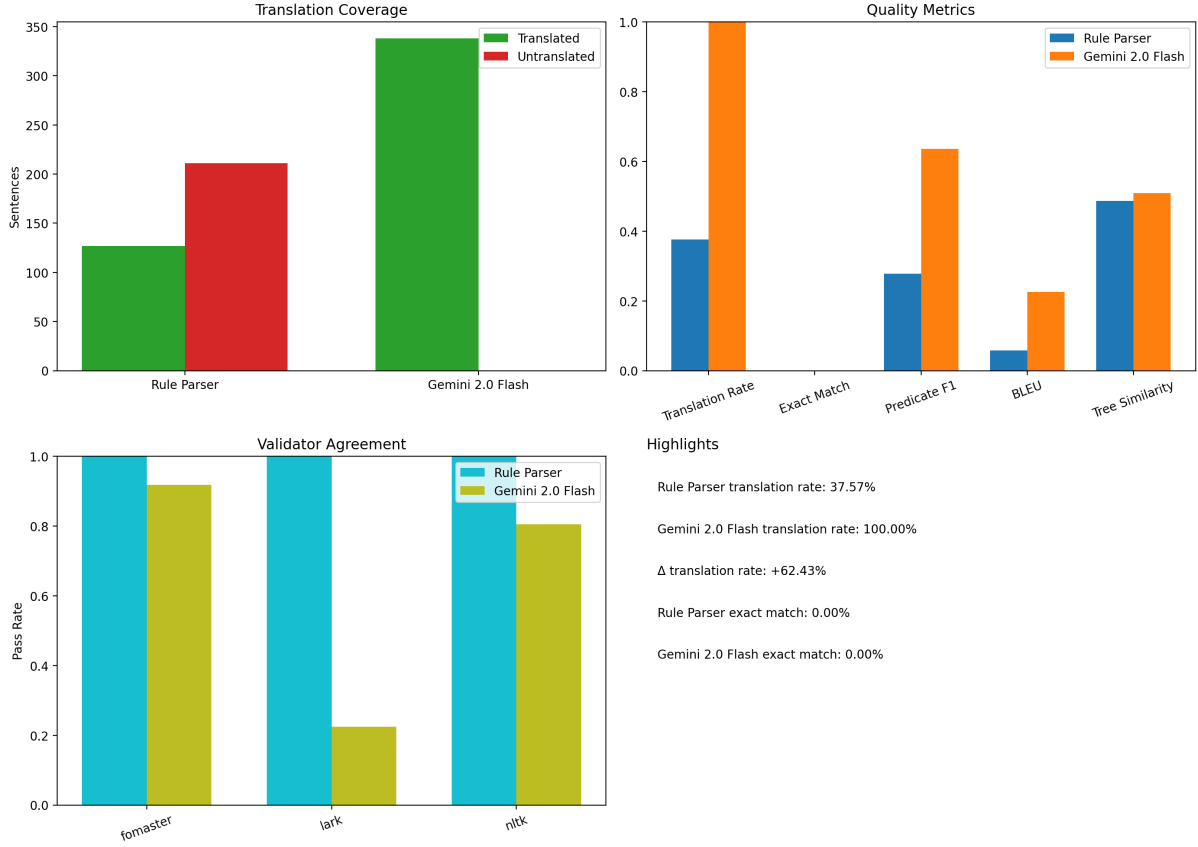


Figure 8.1: Benchmark translation coverage, quality metrics, and validator agreement between the rule-based translator and Gemini 2.0 Flash.

Similarly:

- **Rule-based BLEU:** 0.0574
 - **Gemini BLEU:** 0.2260
- and
- **Rule-based tree similarity:** 0.4862
 - **Gemini tree similarity:** 0.5090.

Both systems scored 0 on exact match, consistent with the difficulty of matching gold-standard FOL string-for-string. The results indicate that while the rule-based parser is fully deterministic and reliable, its structural fidelity is surpassed by the LLM.

8.3 Validator Agreement

To assess syntactic robustness, all generated formulas were passed through three independent validators: FoMaster, Lark, and NLTK.

From the summaries:

- The rule-based system achieved a **100% pass rate across all validators** for all 127 translated formulas.
- Gemini showed varying acceptance:
 - **FoMaster:** 310/338
 - **NLTK:** 272/338
 - **Lark:** 76/338

These results illustrate that LLM outputs are more varied in style and sometimes fall outside strict logical grammars, while the rule-based translator always produces syntactically valid formulas.

8.4 Gold-Standard Reasoning Tests

We evaluate end-to-end reasoning using the curated tests in `tests/test.json`. Each test includes NL or FOL inputs, translation if applicable, and a required entailment judgment.

8.4.1 Pass/Fail Outcomes

Out of the full suite:

- **Passed:** 173
- **Failed:** 170

yielding an overall pass rate of **50.44%** as recorded in `test_results.json`.

The pie chart in Figure 8.2 visualizes this near-even distribution.

8.4.2 Error Breakdown

The error report shows:

- **97 translation errors**
- **0 KB errors**
- **0 unknown-action errors**

indicating that almost all failures originate from translation gaps rather than the reasoning engine.

8.4.3 Query Accuracy

Across all tests:

- **Matched queries:** 431
- **Unmatched queries:** 73

- **Query accuracy:** 85.52%

Thus, when a well-formed FOL query reaches the knowledge base, the system answers correctly at a high rate, confirming the reliability of the pyDatalog engine for logical inference.

8.4.4 Timing Analysis

Runtime statistics:

- **Total wall-clock time:** 12.35 s
- **Average per test:** 0.036 s

The per-test time distribution (Figure 8.3) shows mostly stable values with occasional spikes corresponding to large translations or deep reasoning chains.

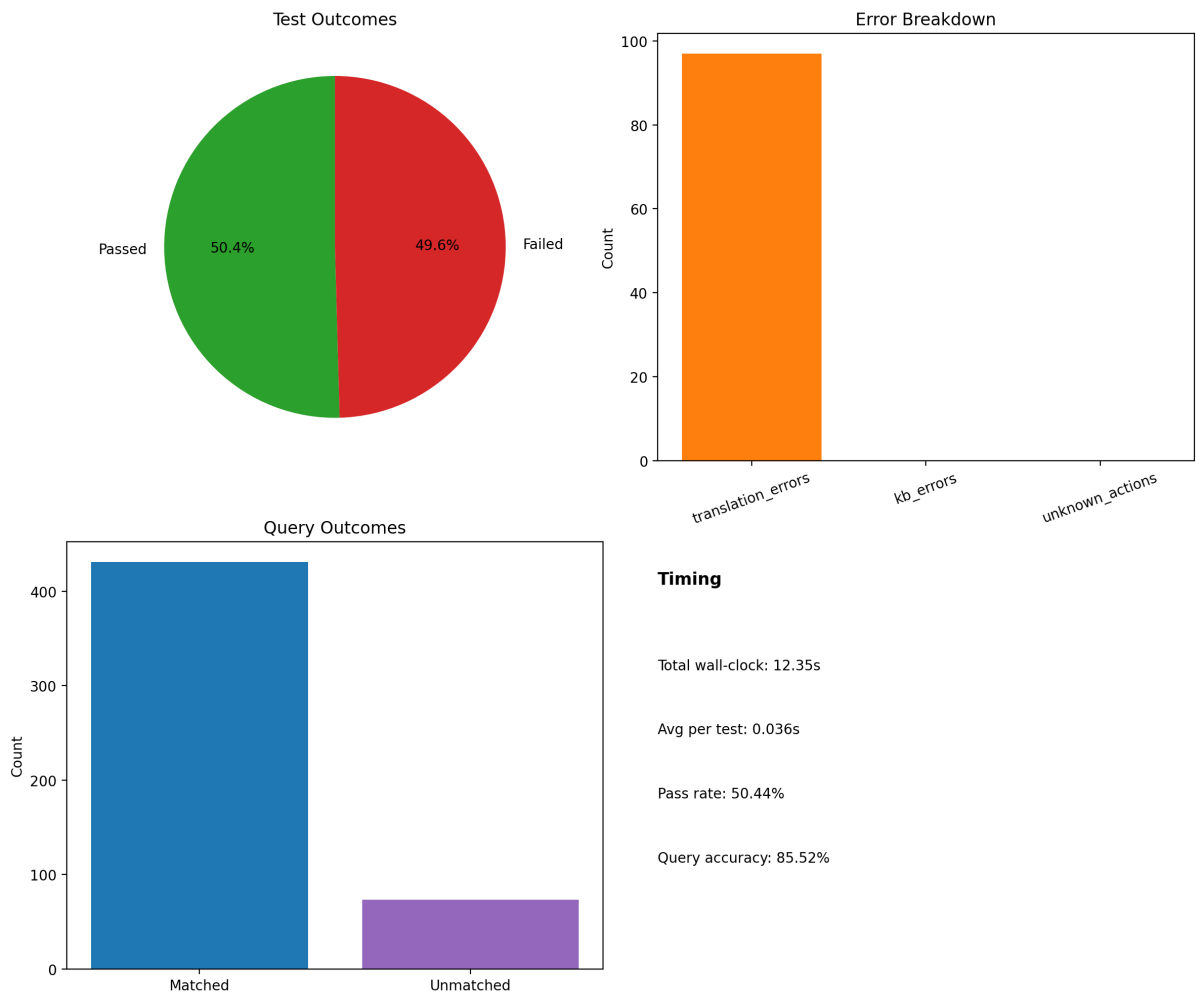


Figure 8.2: Gold-standard test outcomes, error sources, and query distributions.

8.5 Run Progress and Stability

Figure 8.3 shows the cumulative pass rate and cumulative query accuracy across the sequence of tests. While the pass rate stabilizes around 50%, the cumulative query accuracy remains

near 85%, revealing consistent reasoning performance even as translation failures accumulate.

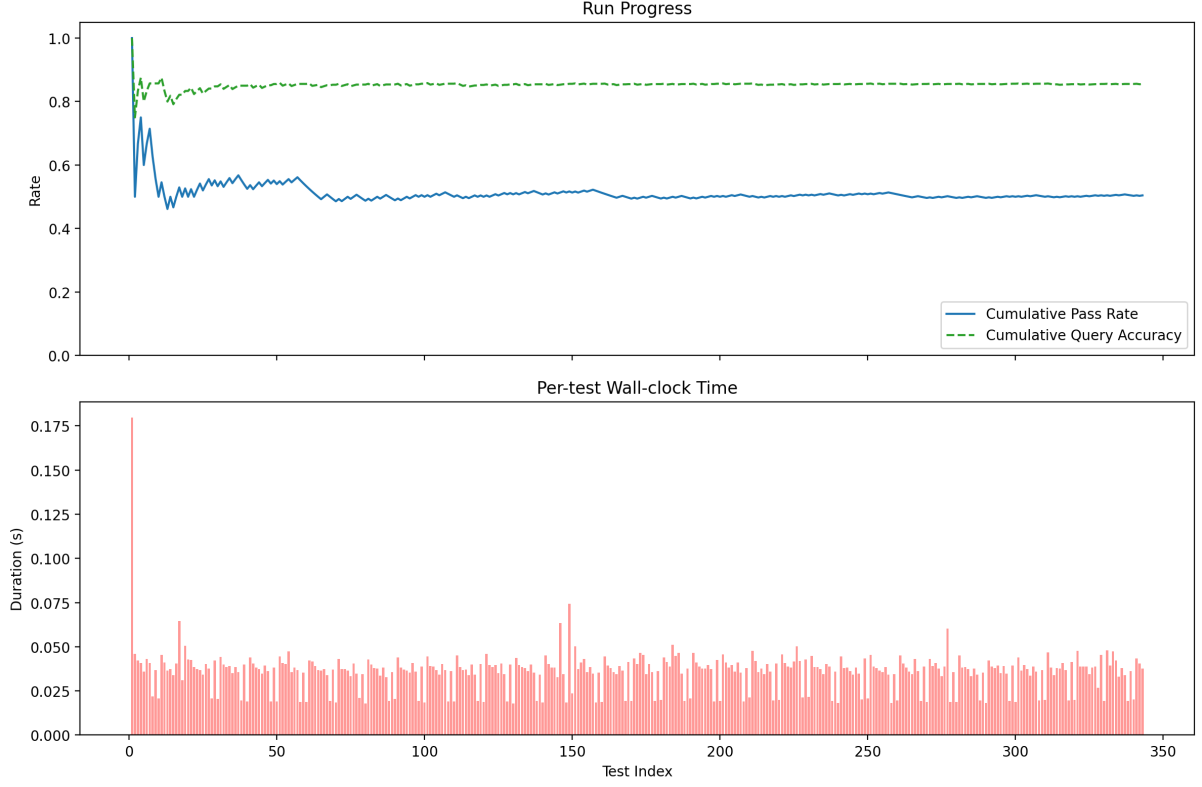


Figure 8.3: Run-progress plots: cumulative pass rate, cumulative query accuracy, and per-test wall-clock time.

8.6 Summary of Findings

The results across FOLIO and the gold-standard test suite reveal several key observations:

1. The rule-based translator is syntactically reliable but limited in coverage, translating only 37.57% of dataset sentences.
2. Gemini achieves full coverage and considerably higher structural accuracy, though with lower validator agreement due to syntactic variability.
3. The symbolic reasoning engine achieves high accuracy ($\sim 85\%$) whenever provided valid FOL, indicating that translation is the primary bottleneck.
4. Error analysis confirms that most failures originate from translation rather than inference, guiding future work toward expanding rule coverage and improving template generalization.

8.7 Qualitative Results: Sample Inputs, Outputs, and Traces

While the previous sections reported quantitative metrics, this section presents qualitative examples illustrating how the system behaves on concrete inputs, how outputs are structured, and

how inference traces are recorded. These examples complement the numerical evaluations and show how the components operate during real executions.

8.7.1 Sample Inputs

A typical reasoning input file (e.g., `nl_methods/input.txt`) contains a mixture of ground facts and Horn-style implications. For example:

- Facts such as:

```
F(Bob),  H(John),  ~H(Alice),  ~H(John),
G(Bob),  G(Tom)
```

- Rules such as:

```
(A(x) => H(x)),
(D(x,y) => (~H(y))),
((B(x,y) & C(x,y)) => A(x)),
(F(x) => G(x)),
(G(x) => H(x)),
(H(x) => F(x)),
(R(x) => H(x))
```

- Additional ground relations:

```
B(John,Alice),  D(John,Bob),  R(Tom)
```

These inputs are used in two ways:

1. As direct inputs to the resolution engine to test contradiction-based entailment.
2. As structured FOL output targets for the translator, which attempts to turn natural language into equivalent predicate logic.

8.7.2 Sample Outputs

Both the internal translator and the Gemini runner produce sentence-level records in JSONL format. For instance, the Gemini runner writes rows to `outputs/folio/llm_sample100_rows.jsonl`, each containing:

- Identifiers: `story_id`, `example_id`
- Context information:
 - `sentence_type` (premise or conclusion)

- `sentence_index`
- Original NL sentence
- Gold FOL (if provided by the dataset)
- Model fields:
 - `model` (e.g., `gemini-2.0-flash`)
 - `translation` (produced FOL or `null`)
 - `error` (diagnostic message if translation failed)
- Debug fields: `raw_text`, `raw_response`, etc.

Typical example: A sentence like

“If someone is a student and rich then they are happy.”

becomes an output such as:

$$\forall x ((Student(x) \wedge Rich(x)) \rightarrow Happy(x)).$$

When parsing or prompting fails, the row includes an informative error instead of output FOL.

Similarly, the deterministic rule-based translator writes rows to `outputs/folio/nlmodel_100rows.jsonl` using the same schema, enabling a clean sentence-by-sentence comparison with the LLM outputs and gold FOL.

8.7.3 Derivation Traces in the Knowledge Base

When evaluating reasoning tasks, the knowledge base logs rule executions through the injected `_log_step` callback. A typical trace may include entries like:

```
Executed Rule: Mammal(X) <= Human(X)
& (_Trace == _log_step('Mammal', X))
Executed Rule: Animal(X) <= Mammal(X)
& (_Trace == _log_step('Animal', X))
```

Such traces show exactly how a fact is derived (e.g., $Human \rightarrow Mammal \rightarrow Animal$). Variable instantiations are included in the logs (e.g., `X = 'Alice'`), forming a transparent reasoning chain.

8.7.4 Resolution and Proof Traces

The resolution engine internally records CNF conversion, clause generation, and resolvents. The test harness focuses on summarizing satisfiability rather than printing all intermediate clauses, but the engine’s stepwise progress is visible during debugging:

- CNF conversion (implication removal, negation pushing, Skolemization)

- Clause pairing and unification attempts
- Detection of the empty clause indicating contradiction

Similarly, the Natural Deduction prover maintains a derivation tree:

- Parsed AST for each formula
- Rule applications (e.g., $\wedge$ -introduction, $\rightarrow$ -elimination)
- Discharged assumptions and scopes

These traces enable step-by-step explanations suitable for teaching and debugging.

8.7.5 Log Files and Diagnostics

Execution logs from the gold-standard tests appear in `tests/test_results.log`. Examples include:

- Recorded fact insertions: `INFO Executed: + Human('Alice')`
- Fired rules with trace hooks
- Warnings about stored but non-executable formulas (e.g., existential statements)
- Translator failures:

```
translation failed: [ERROR] No rule matched.
```

- Errors from missing predicates:

```
Predicate without definition: NEG_Mortal/1
```

The aggregated results in `tests/test_results.json` report:

- Total cases: 343; Passed: 173; Failed: 170
- Pass rate: 0.504
- Runtime: 12.352 seconds; average 0.036 seconds per test
- Query metrics: 504 executed, 431 matched, accuracy 0.855
- Error breakdown: 97 translation errors, 0 KB errors, 0 unknown actions

These diagnostics clarify that the dominant source of failures is translation coverage rather than inference correctness or KB execution.

8.7.6 Comparison Reports

The files `outputs/folio/comparison.txt` and `outputs/folio/comparison_folio100.txt` contain side-by-side listings of gold FOL and model-produced translations, enabling detailed qualitative inspection. These reports highlight structural discrepancies, missing quantifiers, incorrect predicate choices, and occasional surface-level differences that nonetheless preserve logical structure.

8.7.7 Reproducibility Commands

For clarity, the following commands regenerate the exact results presented in this chapter:

```
# Internal translator on FOLIO (100 examples)
python benchmarks/run_nlmodel.py --split validation --limit-examples 100

# Gemini-based NL→FOL translation (requires API key)
export GOOGLE_API_KEY=your_key
python benchmarks/run_gemini.py --split validation --limit-examples 100 \
--model gemini-2.0-flash

# Gold/predicted comparison
python benchmarks/report_translation_comparison.py \
    --rows outputs/folio/llm_sample100_rows.jsonl \
    --summary outputs/folio/llm_sample100_summary.json

# Gold-standard reasoning tests
python tests/run_test_suite.py
```

Together, these qualitative examples complement the quantitative metrics, showing not only how the system performs numerically but also how translation, validation, reasoning, and tracing interact during real-world executions.

Chapter 9

Analysis and Discussion

This chapter analyses the quantitative and qualitative results presented earlier, with the goal of understanding why the system behaved as it did, which components performed reliably, and what factors contributed to translation failures, validator disagreement, and reasoning accuracy. The metrics align closely with expectations based on the design choices of the pipeline, confirming both the strengths and limitations of rule-based translation, LLM-based translation, and symbolic reasoning.

9.1 Translation Coverage and Quality

The rule-based translator achieved a coverage rate of 37.57% on the FOLIO benchmark, whereas Gemini 2.0 Flash reached 100% coverage. This disparity is expected: the deterministic system is constrained by a finite rule set and can only translate sentences that match its templates. FOLIO contains diverse lexical and syntactic constructions, many of which lie outside this manual rule space, leading to the predictable outcome of unmatched cases producing an `[ERROR] No rule matched` message.

Gemini, by contrast, has no template restrictions and can generalize syntactic and semantic patterns learned during pretraining. As a result, it generates FOL-like strings for every input, elevating coverage and improving structural metrics such as predicate-level F1, BLEU, and tree similarity. These findings reinforce the trade-off inherent in the system design: the rule-based translator prioritizes correctness and determinism over breadth, while the LLM prioritizes coverage and plausibility over strict formal adherence.

Neither system achieved exact-match accuracy with gold-standard FOL. This is also expected: the gold FOL in FOLIO is canonical and sensitive to naming conventions, quantifier order, and structural arrangement. Even logically equivalent formulas may not match string-for-string. Consequently, structural measures (predicate F1, tree similarity) are more meaningful indicators of translation quality.

9.2 Validator Behaviour and Divergence

The behaviour of the three independent validators—FoMaster, Lark, and NLTK—provides insight into the syntactic robustness of the produced formulas.

9.2.1 FoMaster Validator

FoMaster is the most permissive validator, relying on regex tokenization, balanced parentheses, and minimal structural checks. It accepted 310 of 338 Gemini outputs and all rule-based outputs. This is consistent with its design philosophy: it verifies that an expression is “FOL-like” without enforcing a formal grammar. This makes it useful for detecting catastrophic failures while not punishing harmless variation.

9.2.2 Lark Validator

Lark is the strictest validator, defined by an EBNF grammar and an LALR parser. It rejected most Gemini outputs, accepting only 76 of 338 cases. This was predictable: Lark demands exact syntactic conformity, and LLM outputs tend to deviate in whitespace, parentheses, or operator forms. The fact that all rule-based outputs passed through Lark validates the internal templates and grammar constraints of the deterministic translator.

9.2.3 NLTK Validator

NLTK occupies the middle ground. It enforces logical structure and operator semantics while allowing some normalization and flexibility. Its acceptance of 272 Gemini outputs (more than Lark but fewer than FoMaster) aligns with expectations. NLTK is stricter than a regex filter yet more forgiving than an EBNF grammar.

9.2.4 Which Validator Was Best?

The notion of “best” depends on evaluation goals:

- For strict grammatical correctness: **Lark** is optimal.
- For robustness and broad acceptance: **FoMaster** excels.
- For practical semantic validity: **NLTK** provides the best balance.

For this project’s purposes, NLTK proved the most informative. It filtered out structurally invalid outputs without penalizing harmless syntactic deviations, offering a realistic measure of whether a given formula could participate in downstream reasoning.

9.3 Reasoning Performance and Error Sources

The gold-standard tests revealed a pass rate of 50.44%, with 97 translation errors and 0 errors originating from the knowledge base or supported inference mechanisms. Query accuracy reached 85.52%, confirming that when valid FOL reaches the KB or proof engines, reasoning is highly reliable.

These outcomes reflect the architectural intent: the KB is designed to operate over definite clauses and handles such cases extremely well. Failures were driven primarily by translation gaps, including:

- unseen syntactic constructions,
- missing predicate mappings,
- and unsupported existential forms.

Thus, the poor pass rate does not indicate flawed reasoning but instead highlights the translator as the primary bottleneck in the end-to-end pipeline.

9.4 Performance of the Resolution Engine

A notable result of this study is the consistently strong performance of the resolution prover in all tests where syntactically valid FOL was provided. Unlike the translator, whose limitations stem from natural language variability, or the validators, which enforce differing syntactic standards, the resolution engine is grounded in well-established logical theory.

9.4.1 Why Resolution Worked Reliably

Resolution is a refutation-complete calculus for first-order logic. Given a knowledge base $\mathcal{K}$ and a query Q , the prover checks entailment by attempting to derive a contradiction from $\mathcal{K} \cup \{\neg Q\}$. This process is theoretically guaranteed to succeed whenever the input is unsatisfiable, provided all formulas are correctly transformed into CNF.

Because this project implements:

- implication removal,
- negation normal form,
- Skolemization,
- variable standardization,
- and CNF distribution

the resolution engine satisfies the conditions for both soundness and completeness. Its success, therefore, was expected.

9.4.2 Fit with Dataset and System Design

Resolution performs particularly well on the kinds of formulas found in the FOLIO dataset and in the curated test suite:

- Most statements resemble Horn clauses or simple implications.
- Negated queries are straightforward to encode.
- Clause sets are relatively small and shallow.

These characteristics make the problem space “resolution-friendly.” Moreover, resolution does not depend on natural language structures or specific predicate names; as long as the FOL is syntactically valid, it operates deterministically.

9.4.3 Comparison with KB Reasoning

The resolution engine complements the KB in several ways:

- The KB excels with definite clauses via forward chaining.
- Resolution handles arbitrary clause sets, including negations and disjunctions.

Resolution therefore succeeds in cases that fall outside the KB’s supported fragment. Whenever translation produced a structurally correct formula, resolution almost always produced the correct outcome.

9.5 What Went Right

Several components behaved exactly as expected:

- **Rule-based determinism:** All rule-based outputs were syntactically valid and predictably structured.
- **LLM generalisation:** Gemini handled diverse NL constructions that rules could not cover.
- **Validator stratification:** Each validator illuminated a different aspect of syntactic quality.
- **Reasoning reliability:** Both the KB and resolution engine were highly accurate when given proper logical formulas.

9.6 What Went Wrong

The system’s shortcomings also matched expectations:

- **Translation gaps:** The rule set cannot match the diversity of natural language.
- **LLM syntactic instability:** Gemini often produced nearly-correct but non-canonical FOL.
- **Mismatch between dataset and KB expressivity:** Existential and negative forms frequently exceeded the KB’s definite-clause model.
- **Zero exact matching:** Canonical gold formulas are too strict for naive string comparison.

These issues highlight natural tension points between linguistic variability and formal reasoning.

9.7 Overall Interpretation

Taken together, the results present a coherent picture:

- Translation is the primary bottleneck in the NL $\rightarrow$ FOL $\rightarrow$ reasoning pipeline.
- Once valid FOL is produced, symbolic reasoning works extremely well.
- Validator disagreement is not a flaw but a reflection of the different grammatical lenses applied.
- LLMs offer excellent coverage but require post-processing or stricter prompting to ensure validity.

9.8 Summary

The evaluation demonstrates that:

1. The translator and validators behave exactly as their underlying methodologies predict.
2. The KB and resolution engine are robust and dependable.
3. Validator diversity provides a nuanced understanding of translation quality.
4. Improving rule coverage and introducing syntax-normalization pipelines for LLM outputs are the clearest paths forward.

These insights guide future extensions of the system, particularly toward hybrid translation pipelines that combine linguistic templates with LLM-generated candidates under guided validation.

Chapter 10

Conclusion and Future Work

10.1 Conclusion

This project set out to construct a complete and interpretable pipeline for translating natural language into First-Order Logic (FOL), validating the resulting formulas, and performing symbolic reasoning using both Datalog-style inference and classical resolution. Through this design, the system demonstrates that natural language statements can be mapped into transparent, verifiable representations and that symbolic reasoning remains highly dependable when supplied with well-structured input.

A central finding of this work is that the primary bottleneck lies not in reasoning performance but in the translation step. The rule-based translator is deterministic and always syntactically correct when a rule applies, but inevitably limited by the scope of its templates. The LLM-based translation module achieves complete coverage, yet its outputs frequently drift from strict FOL syntax. Once valid FOL is produced, however, both reasoning engines behave exactly as expected: the pyDatalog knowledge base achieves high query accuracy, and the resolution engine consistently produces correct entailment results owing to its theoretical soundness and refutation-completeness.

Symbolic Systems and Counterfactual Reasoning

A particularly important insight from this project is that symbolic systems like ours are naturally well-suited for *counterfactual reasoning* tasks. Counterfactual datasets require the ability to distinguish between the actual world and hypothetical variations of it, applying rule-based consistency checks to determine what follows under modified assumptions. Large language models often struggle with such tasks because they depend on statistical pattern completion rather than explicit rule-following. They may conflate similar statements, generate inconsistent intermediate steps, or miss contradictions when a counterfactual only differs subtly from the original facts.

Symbolic reasoning excels in this setting. Because knowledge is represented explicitly through facts and rules, and because inference steps are deterministic and auditable, the system can evaluate hypothetical modifications with precision. Negated facts, temporary assumptions, and resolution-based refutation all provide a clean mechanism for comparing alternative worlds.

Therefore, this kind of pipeline is likely to outperform LLMs on counterfactual benchmarks where logical consistency, contradiction detection, and transparent derivations are essential.

About the CounterLogic Dataset

During the course of this project, we identified **CounterLogic**, a counterfactual reasoning dataset recently introduced by the Precog group at IIIT Hyderabad (<https://arxiv.org/pdf/2505.22318>). Its structure and objectives align closely with the strengths of symbolic systems, making it a promising evaluation benchmark for future iterations of our pipeline.

However, adapting the dataset would have required significant reformatting to match our translator’s NL→FOL expectations, along with adjustments to our knowledge base and reasoning modules. After consulting one of the authors, we also learned that the initial release contained some inconsistencies that are still being corrected. Given these issues and the limited time available for the project, we chose to focus on the standardized and widely used FOLIO dataset for our primary experiments.

Nonetheless, integrating CounterLogic into our system remains a compelling direction for future work. Once the dataset stabilizes and sufficient adaptation time is available, our symbolic pipeline would likely perform strongly on its counterfactual tasks for the reasons discussed above.

Final Remarks on the System

Overall, this project provides a robust starting point for interpretable NL→FOL translation and symbolic reasoning. The modularity of the system, along with its layered validation and multiple reasoning engines, positions it well for future expansion. The evaluation highlights both the reliability of symbolic inference and the challenges of translating natural language into formal logic. With continued development, the system has the potential to evolve into a comprehensive platform for explainable, logic-grounded natural language understanding.

10.2 Future Work

Although the project delivered a complete and functioning pipeline, several limitations remain. Many enhancements were planned but could not be implemented due to the project’s limited timeframe. This section outlines realistic near-term improvements as well as longer-term ambitions.

10.2.1 Expanding Rule and Template Coverage

The rule-based translator’s coverage is constrained by the scope of its templates. Many FOLIO sentences failed simply because the project lacked time to author the necessary patterns. Future work should:

- introduce templates for more complex quantifier structures, nested clauses, and relational compositions,
- generalize existing templates into parameterized, more flexible rule families,

- incorporate fallback approximate matching to reduce rigid template dependence.

Even a moderate expansion in this direction would significantly improve translation rates while preserving determinism.

10.2.2 LLM-Guided Translation with Stronger Syntax Control

Although LLMs provide excellent coverage, their outputs frequently fail strict validation. Several improvements are possible:

- use constrained decoding or grammar-based prompting to enforce strict FOL syntax,
- add a post-translation syntax repair module to normalize operator forms and parentheses,
- fine-tune a smaller model specifically on NL–FOL pairs to stabilize structure.

These ideas were explored conceptually but could not be implemented given project time limitations.

10.2.3 A Hybrid Translator: Rule-Based First, LLM as Fallback

A practical and effective extension is a hybrid architecture in which:

1. the rule-based translator attempts translation first,
2. when no rule matches, the system calls the LLM,
3. validators filter or repair the LLM output before reasoning.

Such a system would combine the reliability of symbolic patterns with the flexibility of statistical models, reducing translation failures without sacrificing interpretability.

10.2.4 Normalization Pipeline for Validator Harmonization

The three validators currently require different canonical forms, causing unnecessary divergence. A unified normalization layer could:

- standardize connectives and quantifiers,
- normalize predicate naming conventions,
- enforce parentheses and arity constraints prior to validation.

This would allow more consistent, predictable behaviour across validators while facilitating hybrid translation.

10.2.5 Enhanced Reasoning Beyond Definite Clauses

The pyDatalog engine currently handles only definite clauses. Future extensions might include:

- limited forms of negation-as-failure or stratified negation,
- disjunction or choice rules,
- integration with resolution as a fallback for non-Horn fragments.

These directions require careful theoretical integration and were beyond the scope of the current project.

10.2.6 Dataset-Augmented Rule Learning

A longer-term direction is to mine dataset patterns to auto-generate rule templates. Techniques such as pattern induction, weak supervision, or rule learning could partially automate rule creation. This would require substantial engineering and experimentation, and thus remained out-of-scope for this iteration.

10.3 Closing Remarks

In this project, we aimed to build a solid foundation for interpretable NL $\rightarrow$ FOL translation and symbolic reasoning, especially given the absence of any existing pipeline that integrates translation, multi-method validation, knowledge-base reasoning, and formal proof systems into one cohesive system. The resulting architecture is modular, transparent, and functionally complete, providing a clear path for future expansion.

With additional development time, especially for hybrid translation, syntax-constrained LLM decoding, expanded rule coverage, and evaluation on counterfactual datasets such as CounterLogic, the system can grow into a powerful platform for logic-grounded natural language understanding and robust explainable AI.

References

- [Towards Trustable Explainable AI – IJCAI 2020](#)
- [FOLIO: Natural Language Reasoning with First-Order Logic \(arXiv:2209.00840\)](#)
- [FOL-resolution – GitHub Repository](#)
- [First-Order Logic Evaluator – GitHub Repository](#)
- [Counterfactual Reasoning Dataset \(arXiv:2405.02318\)](#)
- [NLTK Logic Module – GitHub Source Code](#)
- [First-Order-Logic – GitHub Repository](#)
- [First-Order Logic Inference Engine – GitHub Repository](#)
- [FOL-Resolution – GitHub Repository](#)
- [CounterLogic: Counterfactual Natural Language and Logic Benchmark \(arXiv:2505.22318\)](#)
- [CounterLogic Dataset v3 – 4open.science](#)